



The Self-Healing Pipeline

How error classification eliminated 100% of engineering oversight



Evgeny Nuger • Principal Engineer



02 · CONTEXT

OnsiteIQ at a glance

4B+

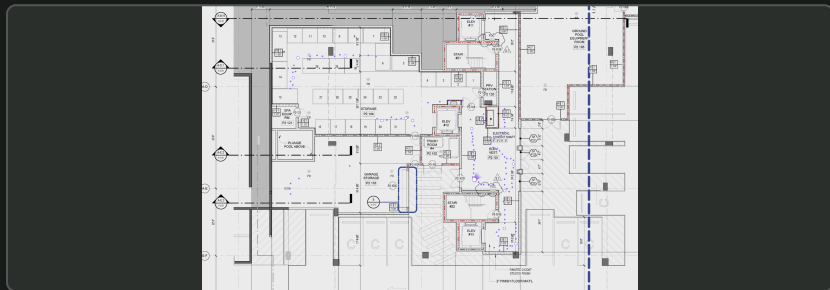
construction-site images processed to date

~10M

images deployed per month, across ~20K datasets

1 engineer

built the migration

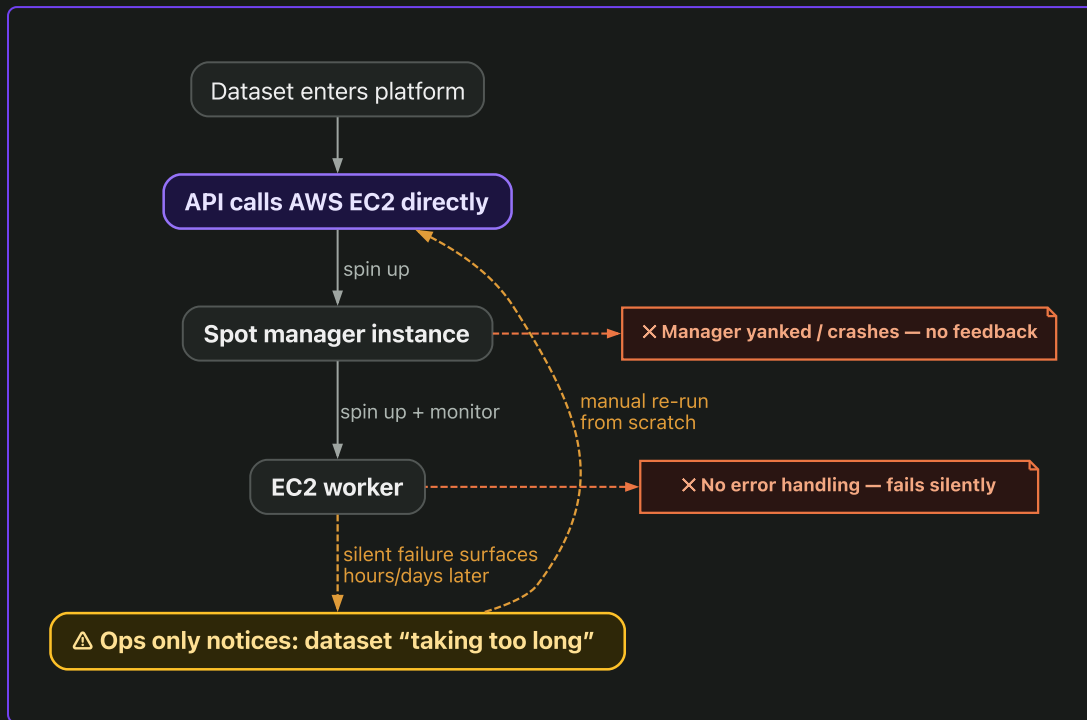


The before state

~20%

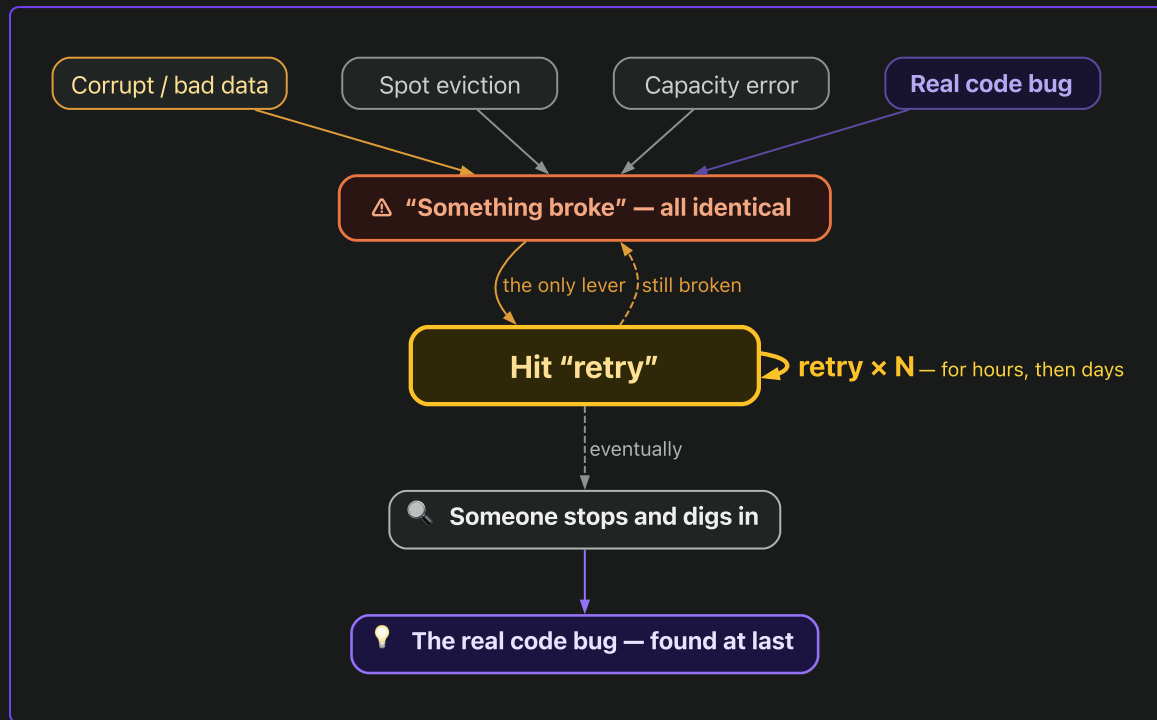
engineering time spent babysitting jobs

- API spun up raw EC2 spot instances
- EC2 Instances managed other spot workers
- No error feedback
- "Taking too long" was the only signal



Noise masked real bugs

Naive retries cleared the symptom and hid the disease — so the same bugs came back for months, with an engineer always in the loop.



One migration solved many problems

Cron + hope

→ **Orchestration**

"Taking too long"

→ **Observability**

Scattered error handling

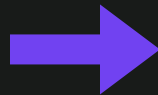
→ **Standardized failure
handling in callbacks**

Silent spot evictions

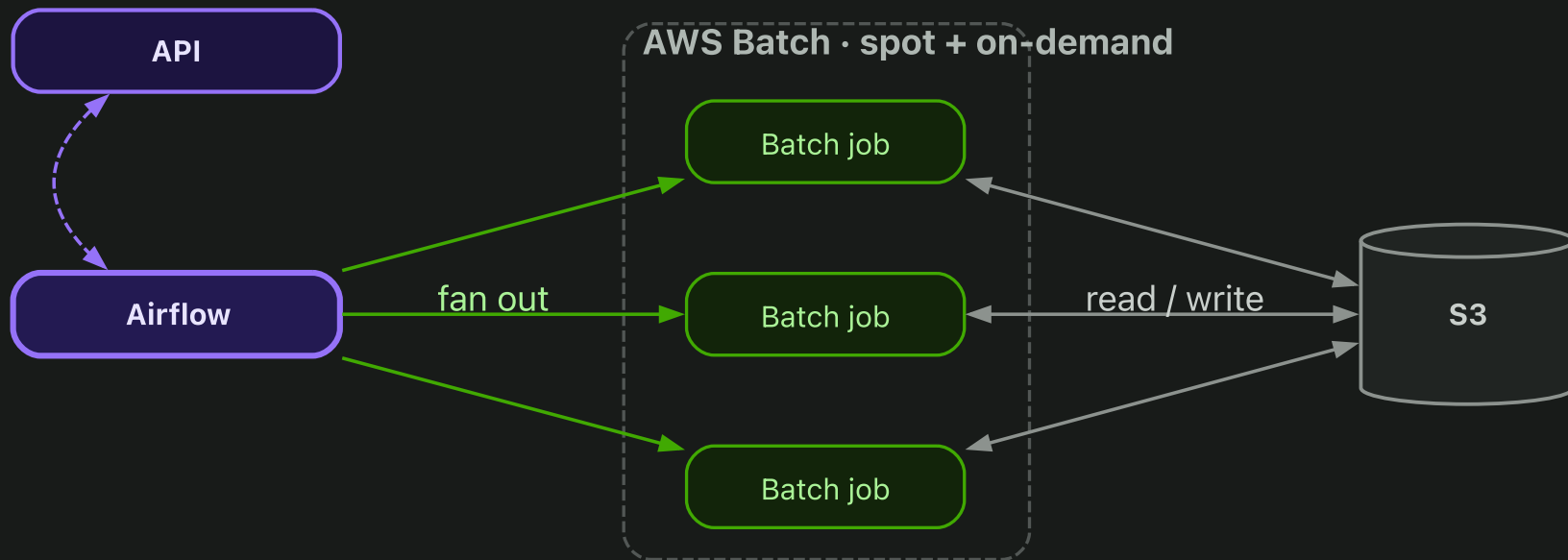
→ **Automatic infra recovery**

Monolithic worker

→ **Decomposition & isolation**



Apache
Airflow



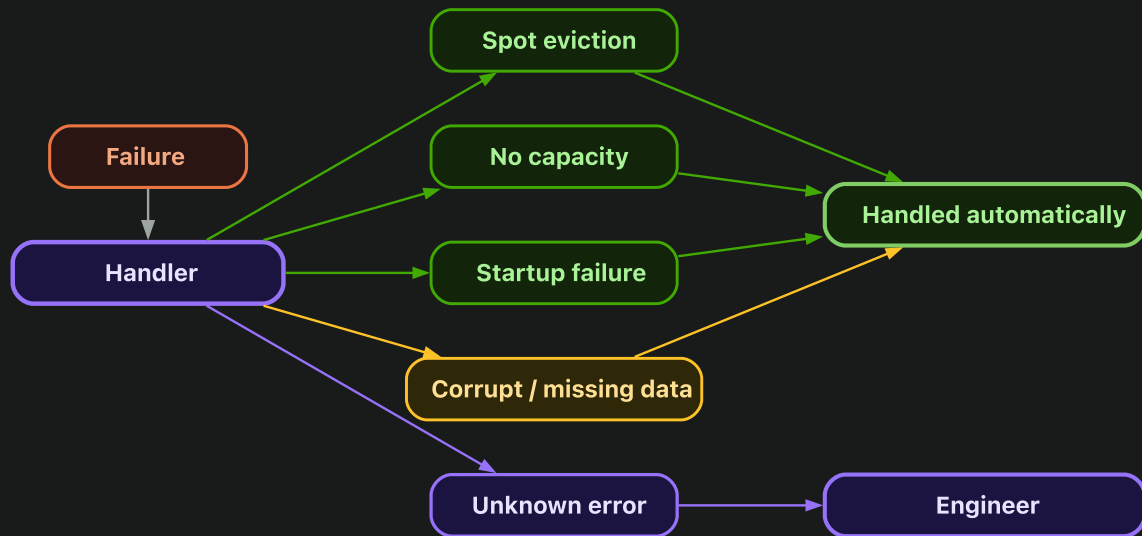
1 engineer

3 months

Sept – Dec 2022

Running 3+ years

Failure Classification



4 of 5
automated

Bad data

Quarantine & automatic review

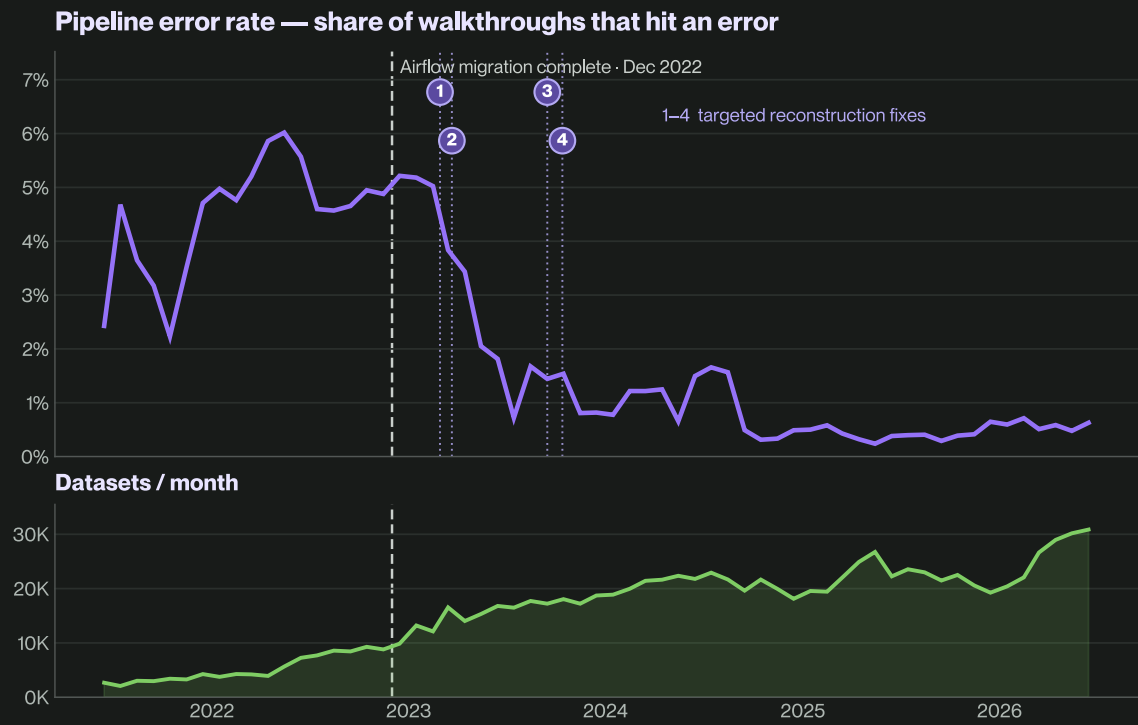
Infrastructure

Resubmit, terminate, reschedule

Real code bug

Ticket assigned to engineer, once

Errors down ~8x, output up ~8x, headache : 0



	> 4B	> 1M
	images deployed to date	datasets all-time
METRIC	BEFORE	AFTER
Pipeline error rate	~5%	<1%
Images / month	1.8M	~15M
Walkthroughs / month	~2.7K	~31K
Eng oversight	~20%	0%
Headcount added	—	0
3+ yrs in prod · 0 headcount changes		

Making any pipeline self-healing in Airflow

Known failures → harden the operator

- Airflow operators are just Python classes; subclass and extend, don't rebuild
- Encode every known, recoverable infra limitation into the operator itself (spot eviction, capacity, cold starts, transient API throttling)
- The operator defends its own work ensuring failures never surface as failures
- Executor-agnostic: same move for Batch, K8s, ECS, Spark

Unknown failures → let the callback decide

- `on_failure_callback` already has the context including exit code, metadata, logs; make it classify, not just notify
 - Classify → tag → route: recover, quarantine, or raise
 - Everything unclassifiable becomes exactly one actionable
 - Reserve native retries for genuinely idempotent transients
-

What's next: Airflow 3

Targeting Q4 2026

1

Self-host on EKS

Leave MWAA for self-managed Airflow 3

Low-Med

2

Batch → EKS workers

Components run as pods, not Batch jobs

Med-High

3

Datasets → records

Process at record level, not per dataset

High

Self-host on EKS + Task SDK

- Leave MWAA for self-managed Airflow 3 on EKS
- Port the custom operators to the Airflow 3 Task SDK
- Sensors + reschedule mode survive, port BaseSensorOperator subclasses
- Move from reschedule-mode sensors to deferrable operators
- The self-healing operator/callback pattern is preserved

Low-Medium

Batch → EKS workers

- Components run as Kubernetes pods instead of AWS Batch jobs
- The spot→on-demand failover re-expressed as node-pools, taints & pod-disruption handling
- GPU node-pools preserved for the detector and feature-extraction components
- ~17 component submissions rewired across the two DAGs

Medium-High

From datasets to records

- Today: one DAG run per dataset wherein components process a whole dataset
- Airflow 3 asset / event-driven scheduling + a queue → per-image / per-node units
- One bad frame no longer gates the dataset; incremental deploy; more parallelism
- A topology redesign, not a port → the biggest conceptual shift

High

THE REAL CHALLENGE

Canary switchover of live incoming data

Route and split datasets between the old Batch deployment and the new EKS one as they arrive, run both in parallel, compare outputs, and cut over per cohort.

WHAT MAKES IT TRACTABLE

The mechanical ports get cheap

Claude Code handles the operator upgrades and DAG refactors, so human effort concentrates on the routing and canary infra and the record-level redesign — not the boilerplate.

Further out

AI IN THE CALLBACK LOOP

The callback stops tagging an engineer

Instead it opens an issue via Claude Code and lets an agent attempt the patch → self-healing extends from infrastructure to code.

USER-FLOW ORCHESTRATION

Bring customer workflows in-house

Use Airflow 3's user-workflow features to run customer-success and operational contact/feedback off today's vendors, on the same platform.

Thank you

Harden your operators, make your callbacks
classify and your orchestrator heals itself too.

Questions welcome.



onsiteiq.io
Product



OnsiteIQ
LinkedIn



Evgeny Nuger
LinkedIn